Critical Analysis of the Opening Range Breakout Strategy on QQQ (2004-2024)

Replicating and Enhancing the Results from
Can Day Trading Really Be Profitable?
by Zarattini & Aziz

Luca Girlando

June 2025

Contents

1 Introduction **2**

2 Baseline Strategy **2**
2.1 Python Implementation 2
2.2 Performance Overview 6
2.3 Comparison with Buy and Hold 7
2.4 Intraday Margin-Neutral Comparison 7
2.5 Conclusion and Limitations 8

3 Adding Stop Loss and Take Profit Logic **8**
3.1 Stop Loss and Take Profit Definition 9
3.2 Python Implementation 9
3.3 Strategy Results: 2016–2023 14
3.4 Comparison with Buy and Hold 15
3.5 Relative Improvements over Baseline 16

4 Position Sizing via Money Management **16**
4.1 Position Sizing Formula 16
4.2 Python Implementation 17
4.3 Backtest Results with Money Management 22
4.4 Comparison with Original Paper 22
4.5 Concluding Remarks 22

5 Final Remarks: Leveraged ETF and Practical Limitations **24**
5.1 Position Sizing and Market Impact 24
5.2 Performance Driven by Money Management 24
5.3 Impact of Slippage 24
5.4 Transaction Costs 25
5.5 Conclusion 26

6 Improving the Strategy: Futures-Based Signal Enhancement **26**
6.1 Trading the Futures Instead of the ETF 26
6.2 Improving the Entry Signal Using Pre-Market Futures Data 27
6.3 Refining the Signal: Focus on the Last Five Minutes Before Open . 28
6.4 Summary 28

7 Conclusion **29**

1 Introduction

This paper critically examines the Opening Range Breakout (ORB) strategy presented in the study *Can Day Trading Really Be Profitable?* by Zarattini & Aziz. The original authors claimed extraordinary returns from $25,000 to over $200,000 in less than 8 years trading the QQQ ETF using a simple ORB approach.

2 Baseline Strategy

We begin our empirical investigation with the baseline Opening Range Breakout (ORB) strategy as proposed in the original paper. The idea is straightforward: define the opening range as the high and low of the first 30 minutes of trading on the QQQ ETF, and enter a long position when the price breaks above the high, or a short position when it drops below the low. All positions are closed before the end of the trading day. This strictly intraday approach ensures zero overnight risk exposure and zero margin requirement, as positions are never held overnight.

2.1 Python Implementation

The following Python code outlines the logic behind the ORB strategy:

Listing 1: ORB Baseline Strategy Implementation

```
# %%
#!/usr/bin/env python3
# -- coding: utf-8 --

import pandas as pd
import datetime
import matplotlib.pyplot as plt

# Load and prepare the dataset
df = pd.read_csv("/Users/Python/Documents/QQQ.csv")

print("Original␣data␣preview:")
print(df.head())


# %%
# Clean and prepare timestamps
```

```
18 df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d␣%
      H:%M:%S%z')
19 df['date'] = pd.to_datetime(df['date'], utc=True).dt.
      tz_convert("US/Eastern").dt.tz_localize(None)
20
21 df['day'] = df['date'].dt.date
22 df['time'] = df['date'].dt.time
23
24 # Check closing time distribution
25 print("\nClosing␣time␣distribution:")
26 time_check = df.groupby("day").last().reset_index()["date"
      ].dt.strftime("%H:%M").value_counts().sort_index()
27 print(time_check)
28
29 # Filter only market hours (up to 15:55)
30 df = df[df['time'] <= datetime.time(15, 55, 0)]
31
32 # Select key candles
33 first_candles = df[df['time'] == pd.Timestamp("09:30:00").
      time()]
34 second_candles = df[df['time'] == pd.Timestamp("09:35:00")
      .time()]
35 last_candles = df.groupby("day").last().reset_index()
36
37 # Plot closing time distribution
38 last_candles['date'].dt.strftime("%H:%M").value_counts().
      sort_index().plot(kind='bar', color='skyblue')
39 plt.title("Distribution␣of␣Closing␣Times")
40 plt.xlabel("Time")
41 plt.ylabel("Count")
42 plt.show()
43
44
45 # %%
46 # Merge dataframes
47 trades = first_candles.merge(second_candles, on="day", how
      ="left", suffixes=("_first", "_second"))
48 trades = trades.merge(last_candles[["date", "day", "close"
      ]], on="day", how="left")
49
```

```
50 # Backtesting logic
51 trades["entry_price"] = trades["open_second"]
52 trades["exit_price"] = trades["close"]
53
54 # Define positions
55 trades["position"] = trades.apply(lambda row: 1 if row["
      close_first"] > row["open_first"]
56                                    else -1 if row["
                                         close_first"] < row["
                                         open_first"]
57                                    else 0, axis=1)
58
59 # Calculate P&L
60 trades["pn1"] = trades["position"] * (trades["exit_price"]
       - trades["entry_price"])
61
62 # Plot cumulative P&L
63 plt.figure(figsize=(12, 6))
64 plt.plot(trades['day'], trades['pn1'].cumsum())
65 plt.title("Cumulative␣P&L␣of␣Strategy")
66 plt.xlabel("Date")
67 plt.ylabel("Cumulative␣P&L")
68 plt.grid(True)
69 plt.show()
70
71 # Print performance statistics
72 avg_trade = trades[trades["position"] != 0]['pn1'].mean()
73 print("\nBacktest␣Results:")
74 print(f"Average␣trade:␣{avg_trade:.4f}")
75 print(f"Total␣number␣of␣trades:␣{len(trades)}")
76 print(f"Number␣of␣long␣trades:␣{len(trades[trades['
      position']␣==␣1])}")
77 print(f"Number␣of␣short␣trades:␣{len(trades[trades['
      position']␣==␣-1])}")
78 print(trades['pn1'].cumsum())
79
80
81 # %%
82 # Compare with Buy-and-Hold (from first available price)
83 trades['strategy_cumulative'] = trades['pn1'].cumsum()
```

```
84 first_price = df[df['date'] == df['date'].min()]['close'].
      values[0]
85 trades['buy_and_hold'] = trades['exit_price'] -
      first_price
86
87 # Plot comparison
88 plt.figure(figsize=(12, 6))
89 plt.plot(trades['day'], trades['strategy_cumulative'],
      label='Trading␣Strategy')
90 plt.plot(trades['day'], trades['buy_and_hold'], label='Buy
      -and-Hold␣QQQ')
91 plt.title("Strategy␣vs␣Buy-and-Hold")
92 plt.xlabel("Date")
93 plt.ylabel("Cumulative␣P&L")
94 plt.legend()
95 plt.grid(True)
96 plt.show()
97
98
99 # %%
100 # Intraday Buy-and-Hold: Buy at 09:30, Sell at EOD Close
101 intraday_entry = df[df['time'] == datetime.time(9, 30)][['
      day', 'open']]
102 intraday_entry = intraday_entry.rename(columns={"open": "
      buy_hold_entry"})
103
104 intraday_exit = df.groupby("day").last().reset_index()[["
      day", "close"]]
105 intraday_exit = intraday_exit.rename(columns={"close": "
      buy_hold_exit"})
106
107 buy_hold = pd.merge(intraday_entry, intraday_exit, on="day
      ", how="inner")
108 buy_hold['buy_hold_return'] = buy_hold['buy_hold_exit'] -
      buy_hold['buy_hold_entry']
109 buy_hold['buy_hold_cumulative'] = buy_hold['
      buy_hold_return'].cumsum()
110
111 # Merge both strategies
112 compare = trades[['day', 'pn1']].copy()
```

```
113 compare = compare.merge(buy_hold[['day', 'buy_hold_return'
      , 'buy_hold_cumulative']], on='day', how='inner')
114 compare['strategy_cumulative'] = compare['pn1'].cumsum()
115
116 # Plot strategy vs intraday buy-and-hold
117 plt.figure(figsize=(12, 6))
118 plt.plot(compare['day'], compare['strategy_cumulative'],
      label='Trading␣Strategy')
119 plt.plot(compare['day'], compare['buy_hold_cumulative'],
      label='Buy-and-Hold␣Intraday', color='green', linestyle
      ='--')
120 plt.title("Strategy␣vs␣Buy-and-Hold␣Intraday")
121 plt.xlabel("Date")
122 plt.ylabel("Cumulative␣P&L")
123 plt.legend()
124 plt.grid(True)
125 plt.show()
```

2.2 Performance Overview

The performance of the strategy from 2004 to 2024 displays a very clear structural change. From 2004 until 2021, the strategy generated consistent profits. However, post-2021, its predictive power and profitability collapse completely.



Figure 1: Cumulative P&L of the Baseline ORB Strategy (2004–2024)

As shown in Figure 1, the strategy performed consistently for over 17 years. But from 2021 onward, the equity curve flattens and reverses, signaling the decay of the

underlying predictive signal.

We further isolate the period considered in the original paper (2016–2023). Trading one share of QQQ per signal (with no transaction costs or slippage), the strategy generated approximately $110 in total profit over 5033 trades. This equates to a mere $0.04 average profit per trade — an extremely narrow edge, highly vulnerable to real-world trading frictions.

2.3 Comparison with Buy and Hold

To put things in perspective, we compare our baseline ORB strategy to a simple Buy and Hold approach — purchasing QQQ and holding it until the end of the backtest window. The result is unsurprisingly superior in terms of total return.

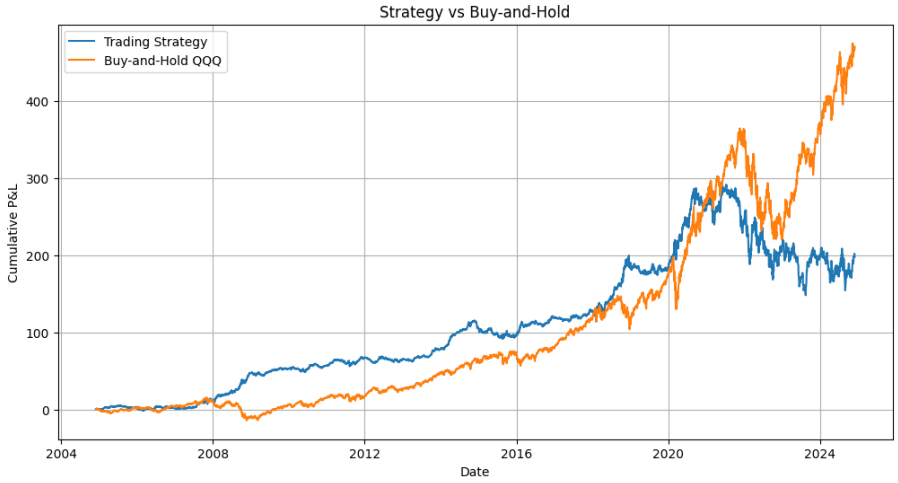


Figure 2: Baseline Strategy vs. Buy and Hold

Figure 2 clearly demonstrates that Buy and Hold dramatically outperforms the baseline strategy in cumulative return. However, it is worth noting that during market crashes such as 2008, 2018, and 2020, our strategy actually generated positive returns while Buy and Hold suffered. This apparent hedging behavior is intriguing — but it did not repeat during the 2022 sell-off.

2.4 Intraday Margin-Neutral Comparison

Institutional trading is typically conducted with margin accounts, which introduces a cost for holding positions overnight. Since our strategy always exits intraday, it avoids these margin costs entirely. A Buy and Hold investor, on the other hand, accumulates overnight margin charges for the entire holding duration.

To address this asymmetry, we compare the ORB strategy with a margin-neutral version of Buy and Hold: buying QQQ at the open each day and selling it at the close — matching the ORB's intraday constraint. The only difference is that this variant loses out on the well-documented "overnight edge" in the Nasdaq.

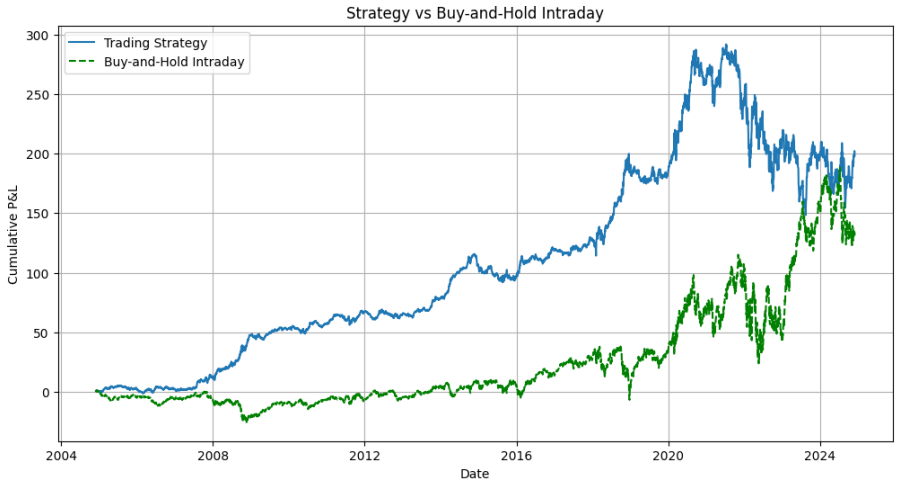


Figure 3: Baseline Strategy vs. Intraday Buy and Hold

As seen in Figure 3, this version of Buy and Hold performs significantly worse than the traditional variant. In this adjusted framework, our baseline ORB strategy outperforms — albeit marginally — highlighting that it captures some exploitable intraday inefficiency. However, this advantage collapses post-2021.

2.5 Conclusion and Limitations

From a quantitative standpoint, the baseline ORB strategy is no longer viable. The deterioration of signal quality post-2021 is severe. A trading edge of four cents per trade is effectively negligible once realistic frictions — such as slippage, spread, and execution latency — are introduced. Even with the margin advantage, the strategy fails to deliver superior risk-adjusted returns over a passive approach.

This leads to a critical observation: if the signal has deteriorated this much, the baseline strategy should arguably be discarded altogether. However, in the spirit of the original paper, we continue by testing additional layers of filters and logical enhancements in the sections that follow.

3 Adding Stop Loss and Take Profit Logic

In this section, we enhance the baseline ORB strategy by implementing a risk management framework based on fixed Stop Loss (SL) and Take Profit (TP) levels.

This is a crucial addition as it transforms the strategy from a purely directional breakout into one that embeds a formal reward-to-risk structure.

3.1 Stop Loss and Take Profit Definition

The Stop Loss is defined as follows:

- For **long positions**, the SL is the **low** of the first 30-minute opening range candle.
- For **short positions**, the SL is the **high** of the first 30-minute candle.

Once the SL is determined, the Take Profit is placed at a level 10 times the risk distance from the entry price. Mathematically:

$$\begin{aligned} R &= |\text{Entry Price} - \text{Stop Loss}| \\ \text{Take Profit} &= \text{Entry Price} + 10 \cdot R \quad \text{(for long positions)} \\ \text{Take Profit} &= \text{Entry Price} - 10 \cdot R \quad \text{(for short positions)} \end{aligned}$$

This structure implements a 1:10 reward-to-risk ratio, ensuring that large winning trades can offset a significant number of small losses.

3.2 Python Implementation

The following Python code block shows how this Stop Loss and Take Profit logic can be integrated into the ORB framework:

Listing 2: ORB Strategy with Stop Loss and Take Profit

```
# %%
import pandas as pd
import datetime as datetime
from tqdm import tqdm
import matplotlib.pyplot as plt

class trader():
    def _init_(self):
        self.in_buy = 0
        self.in_sell = 0
        self.buy_price = 0
        self.sell_price = 0
        self.stop_loss = 0
```

```
14          self.take_profit = 0
15          self.trades = []
16          self.pnl = [0]
17          self.dates = []
18          self.buy_prices = []
19          self.sell_prices = []
20          self.fee = 0
21          self.a = 25000
22          self.account = [self.a]
23          self.r = 0
24          self.opening = 0
25          self.n_list = []
26          self.notionals = [0]
27          self.unit_trades = []
28
29      def reset(self):
30          self.in_buy = 0
31          self.in_sell = 0
32          self.buy_price = 0
33          self.sell_price = 0
34          self.stop_loss = 0
35          self.take_profit = 0
36
37      def close_pos(self, date):
38          self.n = 1
39          self.notionals.append(self.n*self.opening)
40          self.trades.append((self.sell_price - self.
               buy_price)*self.n - 2*self.n*self.fee)
41          self.unit_trades.append(self.trades[-1]/self.n)
42          self.a = self.a +self.trades[-1]
43          self.account.append(self.a)
44          self.pnl.append(self.pnl[-1] + self.trades[-1])
45          self.dates.append(date)
46          self.buy_prices.append(self.buy_price)
47          self.sell_prices.append(self.sell_price)
48
49  df = pd.read_csv("/Users/Python/Documents/QQQ.csv")
50
51  df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d␣%
       H:%M:%S%z')
```

```
52
53 df['date'] = pd.to_datetime(df['date'], utc=True).dt.
      tz_convert('US/Eastern').dt.tz_localize(None)
54
55 df['day'] = df['date'].dt.date
56 df['time'] = df['date'].dt.time
57
58 df = df[df['day'] >= datetime.date(2016,1,1)]
59 df = df[df['day'] <= datetime.date(2023,2,17)]
60
61 df = df[df['time'] <= datetime.time(15,55,0)]
62
63
64 # %%
65 df_copy = df.copy()
66
67 traderone = trader()
68
69 for t in tqdm(range(1,len(df))):
70     if df['day'].iloc[t] != df['day'].iloc[t-1]:
71         if traderone.in_buy == 1 and traderone.in_sell ==
              0:
72             traderone.in_sell = 1
73             traderone.sell_price = df['close'].iloc[t-1]
74             #print(df['date'].iloc[t-1], "CLOSE BUY", df['
                  close'].iloc[t-1])
75         elif traderone.in_sell == 1 and traderone.in_buy
              == 0:
76             traderone.in_buy = 1
77             traderone.buy_price = df['close'].iloc[t-1]
78             #print(df['date'].iloc[t-1], "CLOSE SELL",df['
                  close'].iloc[t-1])
79         if traderone.in_buy == 1 and traderone.in_sell ==
              1:
80             #print(df['date'].iloc[t-1], "CLOSE POSITION",
                   traderone.buy_price, traderone.sell_price)
81             traderone.close_pos(df['day'].iloc[t-1])
82             traderone.reset()
83
84     if traderone.in_buy == 1 and traderone.in_sell == 0:
```

```
85          if df['low'].iloc[t] <= traderone.stop_loss:
86              traderone.in_sell = 1
87              traderone.sell_price = traderone.stop_loss
88              #print(df['date'].iloc[t], "BUY STOP LOSS", df
                    ['low'].iloc[t], traderone.stop_loss)
89          elif df['high'].iloc[t] >= traderone.take_profit:
90              traderone.in_sell = 1
91              traderone.sell_price = traderone.take_profit
92              #print(df['date'].iloc[t], "BUY TP ", df['high
                    '].iloc[t], traderone.take_profit)
93      if traderone.in_buy == 0 and traderone.in_sell == 1:
94          if df['high'].iloc[t] >= traderone.stop_loss:
95              traderone.in_buy = 1
96              traderone.buy_price = traderone.stop_loss
97              #print(df['date'].iloc[t], "SELL STOP LOSS",
                    df['high'].iloc[t], traderone.stop_loss)
98          elif df['low'].iloc[t] <= traderone.take_profit:
99              traderone.in_buy = 1
100             traderone.buy_price = traderone.take_profit
101             #print(df['date'].iloc[t], "SELL TP ", df['low
                    '].iloc[t], traderone.take_profit)
102
103     if df['date'].iloc[t].time() == datetime.time(9,30,0):
104         if df['open'].iloc[t] < df['close'].iloc[t]:
105             traderone.in_buy = 1
106             traderone.buy_price = df['open'].iloc[t+1]
107             traderone.stop_loss = df['low'].iloc[t]
108             traderone.r = traderone.buy_price - traderone.
                    stop_loss
109             traderone.opening = df['open'].iloc[t+1]
110             traderone.take_profit = round(10*(traderone.r)
                     + traderone.buy_price, 2)
111             #print(df['date'].iloc[t], "buy price:",
                    traderone.buy_price, "stop loss:",
                    traderone.stop_loss, "take profit:",
                    traderone.take_profit)
112         elif df['open'].iloc[t] > df['close'].iloc[t]:
113             traderone.in_sell = 1
114             traderone.sell_price = df['open'].iloc[t+1]
115             traderone.stop_loss = df['high'].iloc[t]
```

```
116             traderone.r = traderone.stop_loss - traderone.
                    sell_price
117             traderone.opening = df['open'].iloc[t+1]
118             traderone.take_profit = round(10*(-traderone.r
                    ) + traderone.sell_price, 2)
119             #print(df['date'].iloc[t], "sell price:",
                    traderone.sell_price, "stop loss:",
                    traderone.stop_loss, "take profit:",
                    traderone.take_profit)
120
121
122 # %%
123 # Primo plot: Andamento dell'account
124 plt.figure(figsize=(10, 5))
125 plt.plot(traderone.dates, traderone.account[1:], label="
       Account␣Value", color='blue')
126 plt.xlabel("Date")
127 plt.ylabel("Account␣Value")
128 plt.title("Account␣Value␣Over␣Time")
129 plt.legend()
130 plt.grid()
131 plt.show()
132
133
134 # %%
135 # Secondo plot: Profit and Loss della strategia
136 plt.figure(figsize=(10, 5))
137 plt.plot(traderone.dates, traderone.pnl[1:], label="
       Strategy␣PnL", color='green')
138 plt.xlabel("Date")
139 plt.ylabel("PnL")
140 plt.title("Strategy␣Profit␣and␣Loss")
141 plt.legend()
142 plt.grid()
143 plt.show()
144
145
146 # %%
147 # Buy & Hold
148 df['trade'] = df['close'] - df['open'].iloc[0]
```

```
149
150 # Terzo plot: Confronto tra strategia e buy & hold
151 plt.figure(figsize=(10, 5))
152 plt.plot(traderone.dates, traderone.pnl[1:], label="
       Strategy␣PnL", color='green')
153 plt.plot(df['date'], df['trade'], label="Buy␣&␣Hold␣Pnl",
       color='red', linestyle="dashed")
154 plt.xlabel("Date")
155 plt.ylabel("PnL")
156 plt.title("Strategy␣vs␣Buy␣&␣Hold␣PnL")
157 plt.legend()
158 plt.grid()
159 plt.show()
```

3.3 Strategy Results: 2016–2023

We backtest the strategy using one share per trade, with an initial capital of $25,000 and no transaction costs or slippage. At the end of the period, the account value reaches $25,120 — a total profit of just $120. The cumulative P&L is shown in Figure 4.

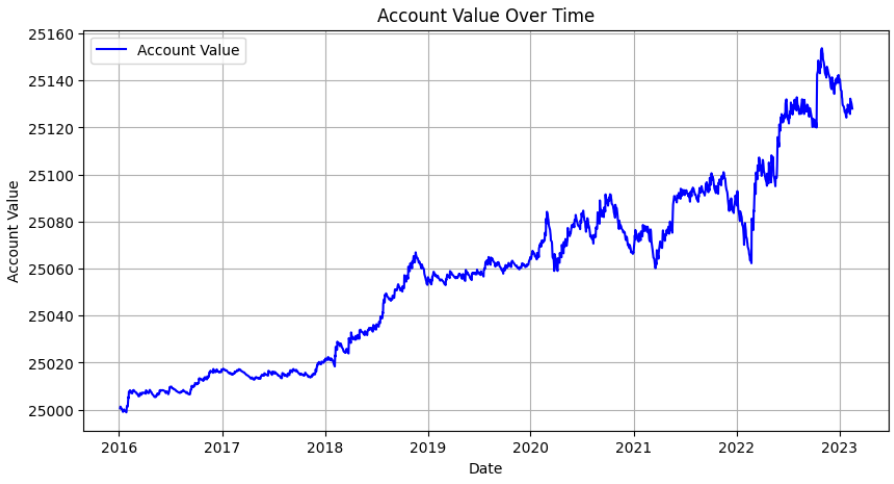


Figure 4: Cumulative Account Value: ORB with Stop Loss and Take Profit (2016–2023)

Isolating the P&L curve alone (ignoring compounding effects on capital), we obtain the following:

Although the profit is marginal, the equity curve appears smoother and more linear than in the baseline case. This suggests a more stable reward structure and



Figure 5: Isolated P&L: SL/TP Strategy (No compounding)

better control of downside risk.

3.4 Comparison with Buy and Hold

Even with the improved risk management framework, the strategy underperforms relative to a passive Buy and Hold of QQQ over the same period. The cumulative performance comparison is presented in Figure 6.



Figure 6: ORB SL/TP Strategy vs. Buy and Hold

Despite being less volatile and structurally more robust, the SL/TP-enhanced ORB strategy fails to generate superior returns compared to holding QQQ continuously.

3.5 Relative Improvements over Baseline

Relative to the baseline strategy, however, the addition of Stop Loss and Take Profit offers some notable advantages:

- The equity curve is significantly more linear.
- Maximum drawdowns are reduced.
- Profit distribution becomes more right-skewed (due to 10R wins).

Nevertheless, the overall profit remains extremely modest, and the lack of robustness after 2021 continues to be a major concern. Without further filters or dynamic logic, this strategy is still far from institutionally viable. The next section explores whether additional signal conditions can meaningfully improve predictive power and profitability.

4 Position Sizing via Money Management

The final enhancement to the strategy, as suggested in the reference paper, involves dynamic position sizing through a money management (MM) rule. So far, each trade has been executed with a fixed position of one share. However, this approach does not account for capital growth or varying trade risk. A more realistic and scalable strategy must allocate capital per trade as a function of the account size and risk per trade.

4.1 Position Sizing Formula

Let A be the current account balance, R be the dollar risk of the trade (i.e., the distance between entry and stop loss), and P the entry price. The number of shares S to be purchased is defined as:

$$S = \left\lfloor \min\left(\frac{0.01 \cdot A}{R}, \frac{4 \cdot A}{P}\right) \right\rfloor \tag{1}$$

This formula is composed of two terms:

- $\frac{0.01 \cdot A}{R}$ ensures that the maximum capital at risk per trade does not exceed 1% of the account.
- $\frac{4 \cdot A}{P}$ accounts for the leverage constraint: the authors assume a broker allows a maximum leverage of 4:1.

The minimum of these two terms is taken to ensure that we do not breach the leverage constraint while maintaining a disciplined risk-per-trade policy. Finally, the result is floored to the nearest integer, as we assume that fractional share trading is not allowed.

4.2 Python Implementation

Below is the placeholder for the Python code that implements the above position sizing logic. Before applying the MM rule, we modify the strategy to maintain a constant notional exposure and visualize the impact on lot sizing. Then, we replace that with the dynamic rule.

Listing 3: ORB Strategy with Dynamic Position Sizing

```
# %%
import pandas as pd
import datetime as datetime
from tqdm import tqdm
import matplotlib.pyplot as plt

class trader():
    def _init_(self):
        self.in_buy = 0
        self.in_sell = 0
        self.buy_price = 0
        self.sell_price = 0
        self.stop_loss = 0
        self.take_profit = 0
        self.trades = []
        self.pnl = [0]
        self.dates = []
        self.buy_prices = []
        self.sell_prices = []
        self.fee = 0
        self.a = 25000
        self.account = [self.a]
        self.r = 0
        self.opening = 0
        self.n_list = []
        self.notionals = [0]
        self.unit_trades = []
```

```
28          self.n_list = []
29
30      def reset(self):
31          self.in_buy = 0
32          self.in_sell = 0
33          self.buy_price = 0
34          self.sell_price = 0
35          self.stop_loss = 0
36          self.take_profit = 0
37
38      def close_pos(self, date):
39          self.n = int(min(self.a*0.01/self.r, 4*self.a/self
               .opening))
40          self.n_list.append(self.n)
41          self.notionals.append(self.n*self.opening)
42          self.trades.append((self.sell_price - self.
               buy_price)*self.n - 2*self.n*self.fee)
43          self.unit_trades.append(self.trades[-1]/self.n)
44          self.a = self.a +self.trades[-1]
45          self.account.append(self.a)
46          self.pnl.append(self.pnl[-1] + self.trades[-1])
47          self.dates.append(date)
48          self.buy_prices.append(self.buy_price)
49          self.sell_prices.append(self.sell_price)
50
51  df = pd.read_csv("/Users/Python/Documents/QQQ.csv")
52
53  df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d␣%
       H:%M:%S%z')
54
55  df['date'] = pd.to_datetime(df['date'], utc=True).dt.
       tz_convert('US/Eastern').dt.tz_localize(None)
56
57  df['day'] = df['date'].dt.date
58  df['time'] = df['date'].dt.time
59
60  df = df[df['day'] >= datetime.date(2016,1,1)]
61  df = df[df['day'] <= datetime.date(2023,2,17)]
62
63  df = df[df['time'] <= datetime.time(15,55,0)]
```

```
64
65
66 # %%
67 df_copy = df.copy()
68
69 traderone = trader()
70
71 for t in tqdm(range(1,len(df))):
72     if df['day'].iloc[t] != df['day'].iloc[t-1]:
73         if traderone.in_buy == 1 and traderone.in_sell ==
              0:
74             traderone.in_sell = 1
75             traderone.sell_price = df['close'].iloc[t-1]
76             #print(df['date'].iloc[t-1], "CLOSE BUY", df['
                  close'].iloc[t-1])
77         elif traderone.in_sell == 1 and traderone.in_buy
              == 0:
78             traderone.in_buy = 1
79             traderone.buy_price = df['close'].iloc[t-1]
80             #print(df['date'].iloc[t-1], "CLOSE SELL",df['
                  close'].iloc[t-1])
81         if traderone.in_buy == 1 and traderone.in_sell ==
              1:
82             #print(df['date'].iloc[t-1], "CLOSE POSITION",
                   traderone.buy_price, traderone.sell_price)
83             traderone.close_pos(df['day'].iloc[t-1])
84             traderone.reset()
85
86     if traderone.in_buy == 1 and traderone.in_sell == 0:
87         if df['low'].iloc[t] <= traderone.stop_loss:
88             traderone.in_sell = 1
89             traderone.sell_price = traderone.stop_loss
90             #print(df['date'].iloc[t], "BUY STOP LOSS", df
                  ['low'].iloc[t], traderone.stop_loss)
91         elif df['high'].iloc[t] >= traderone.take_profit:
92             traderone.in_sell = 1
93             traderone.sell_price = traderone.take_profit
94             #print(df['date'].iloc[t], "BUY TP ", df['high
                  '].iloc[t], traderone.take_profit)
95     if traderone.in_buy == 0 and traderone.in_sell == 1:
```

```
96          if df['high'].iloc[t] >= traderone.stop_loss:
97              traderone.in_buy = 1
98              traderone.buy_price = traderone.stop_loss
99              #print(df['date'].iloc[t], "SELL STOP LOSS",
                    df['high'].iloc[t], traderone.stop_loss)
100         elif df['low'].iloc[t] <= traderone.take_profit:
101             traderone.in_buy = 1
102             traderone.buy_price = traderone.take_profit
103             #print(df['date'].iloc[t], "SELL TP ", df['low
                    '].iloc[t], traderone.take_profit)
104
105     if df['date'].iloc[t].time() == datetime.time(9,30,0):
106         if df['open'].iloc[t] < df['close'].iloc[t]:
107             traderone.in_buy = 1
108             traderone.buy_price = df['open'].iloc[t+1]
109             traderone.stop_loss = df['low'].iloc[t]
110             traderone.r = traderone.buy_price - traderone.
                    stop_loss
111             traderone.opening = df['open'].iloc[t+1]
112             traderone.take_profit = round(10*(traderone.r)
                     + traderone.buy_price, 2)
113             #print(df['date'].iloc[t], "buy price:",
                    traderone.buy_price, "stop loss:",
                    traderone.stop_loss, "take profit:",
                    traderone.take_profit)
114         elif df['open'].iloc[t] > df['close'].iloc[t]:
115             traderone.in_sell = 1
116             traderone.sell_price = df['open'].iloc[t+1]
117             traderone.stop_loss = df['high'].iloc[t]
118             traderone.r = traderone.stop_loss - traderone.
                    sell_price
119             traderone.opening = df['open'].iloc[t+1]
120             traderone.take_profit = round(10*(-traderone.r
                    ) + traderone.sell_price, 2)
121             #print(df['date'].iloc[t], "sell price:",
                    traderone.sell_price, "stop loss:",
                    traderone.stop_loss, "take profit:",
                    traderone.take_profit)
122
123
```

```
124 # %%
125 # Primo plot: Andamento dell'account
126 plt.figure(figsize=(10, 5))
127 plt.plot(traderone.dates, traderone.account[1:], label="
       Account␣Value", color='blue')
128 plt.xlabel("Date")
129 plt.ylabel("Account␣Value")
130 plt.title("Account␣Value␣Over␣Time")
131 plt.legend()
132 plt.grid()
133 plt.show()
134
135 # %%
136 plt.plot(traderone.n_list)
137
138 # %%
139 # Secondo plot: Profit and Loss della strategia
140 plt.figure(figsize=(10, 5))
141 plt.plot(traderone.dates, traderone.pnl[1:], label="
       Strategy␣PnL", color='green')
142 plt.xlabel("Date")
143 plt.ylabel("PnL")
144 plt.title("Strategy␣Profit␣and␣Loss")
145 plt.legend()
146 plt.grid()
147 plt.show()
148
149 # %%
150 """Log graph"""
151 plt.figure(figsize=(10, 5))
152 plt.plot(traderone.dates, traderone.account[1:], label="
       Account␣Value", color='blue')
153 plt.yscale('log')
154 plt.xlabel("Date")
155 plt.ylabel("Account␣Value␣(log␣scale)")
156 plt.title("Account␣Value␣Over␣Time␣(Logarithmic␣Scale)")
157 plt.legend()
158 plt.grid(True, which="both", ls="--")
159 plt.show()
```

4.3 Backtest Results with Money Management

Applying the MM rule over the period 2016–2023 significantly impacts both the performance and the behavior of the strategy. The account, which previously remained almost flat around $25,000, now grows to over $175,000, as shown in Figure 7.

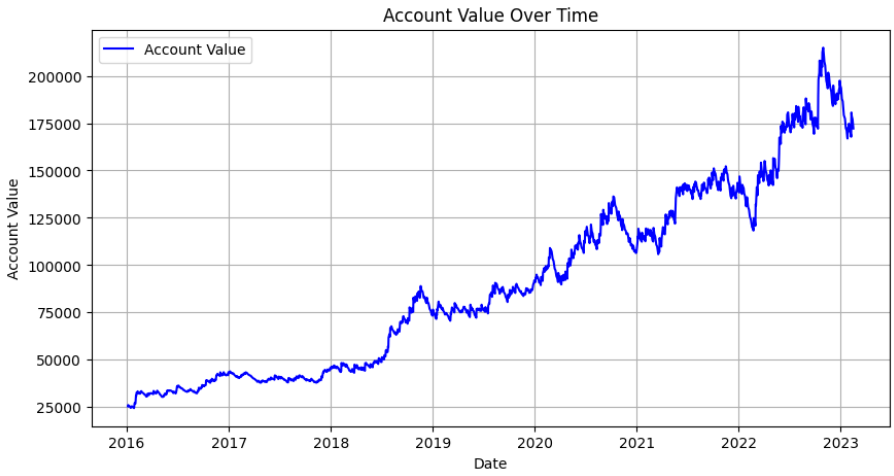


Figure 7: Account Value Over Time with Dynamic Position Sizing

The number of shares purchased per trade now follows a highly variable and generally increasing trajectory, influenced directly by the account value and the volatility of the entry candle (which impacts R). These swings in lot size are illustrated in Figure 8.

4.4 Comparison with Original Paper

Although the absolute account growth differs slightly from that shown in the original paper, this is largely due to differences in scale and presentation. The authors of the paper employ a logarithmic scale for their equity curve. Once we adopt the same representation, the resulting shape aligns closely with their published results, as illustrated in Figure 9.

Minor discrepancies are attributed to rounding (since we do not use fractional shares), slight implementation differences, and a few missing trades.

4.5 Concluding Remarks

This section demonstrates that dynamic position sizing based on account balance and risk dramatically improves capital growth and return scalability. However, the sharp fluctuations in traded volume underscore the sensitivity of the method to

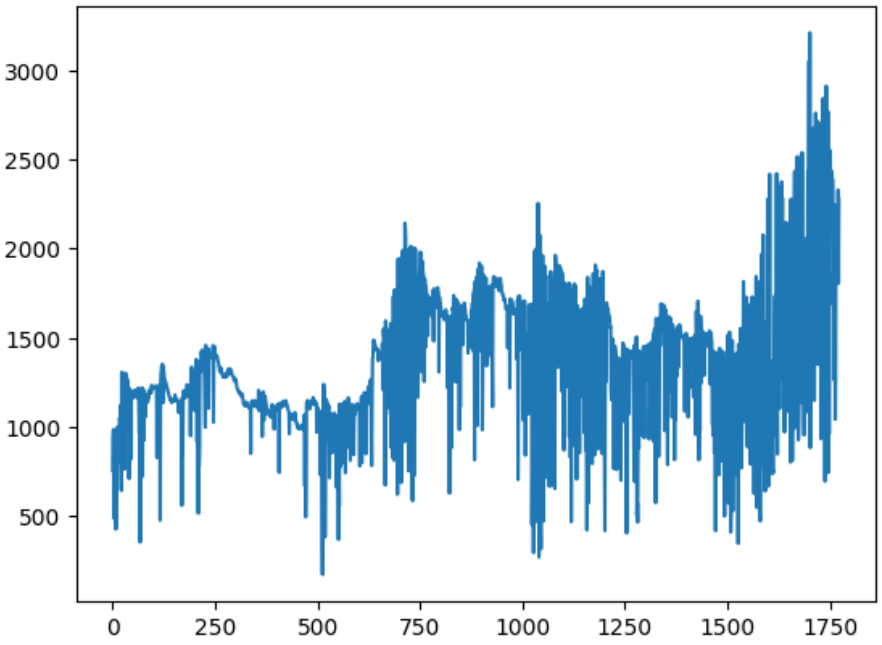


Figure 8: Number of Shares Traded per Entry with MM Rule

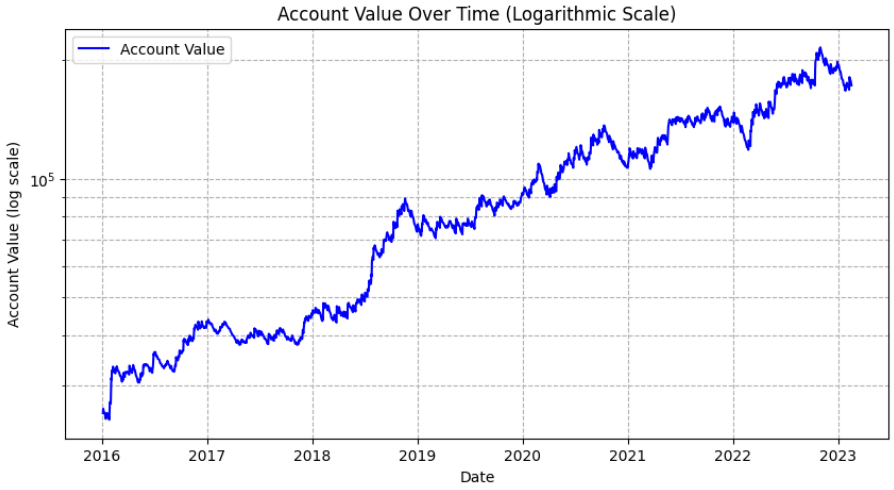


Figure 9: Account Value with Logarithmic Scale

short-term volatility, reinforcing the importance of consistent entry conditions and robust stop placement.

The next step involves evaluating the full strategy across different market regimes, and comparing its risk-adjusted metrics (e.g., Sharpe ratio, drawdown, expectancy) to benchmark strategies and passive investing.

5 Final Remarks: Leveraged ETF and Practical Limitations

To conclude their study, the authors extend their strategy to a leveraged ETF: **QQQT**, a 3x-leveraged version of the NASDAQ-tracking QQQ. This choice circumvents the 4x broker leverage constraint previously imposed in the money management formula and seeks to further amplify gains.

5.1 Position Sizing and Market Impact

While the money management rule accelerates account growth, it also results in exponentially growing position sizes. After eight years of compounding, the strategy suggests trading over 3,200 shares of QQQ per day, corresponding to approximately $800,000 notional per position. Although QQQ is highly liquid and actively traded, its order book shows an average of only 1,000 to 2,000 shares available at the best bid/ask levels at market open.

Attempting to buy or sell such large sizes via market orders at 9:35 AM introduces **slippage** — the execution price deviates from the expected entry due to insufficient volume at the top of the book. Orders are filled at less favorable prices, potentially at the second or third book level. For QQQ, this means an average deterioration of at least one cent per share for trades exceeding top-level liquidity.

5.2 Performance Driven by Money Management

A deeper analysis reveals that the strategy's performance is almost entirely driven by the money management rule and the implicit leverage. The average unitary trade gain is merely $0.07 per share. This is extremely low, especially in the presence of execution costs. A strategy with such narrow edge is particularly vulnerable to frictions.

5.3 Impact of Slippage

To evaluate the real-world viability, we simulate the impact of conservative slippage assumptions:

- Entry slippage: 2 cents per share (due to crossing the spread at 9:35 AM market order entry)
- Stop loss slippage: 4 cents per share (market order exits into volatility)

- Take profit slippage: assumed 0, as limit orders are used

Once these frictions are introduced, the strategy's profitability vanishes entirely, as shown in Figure 10.

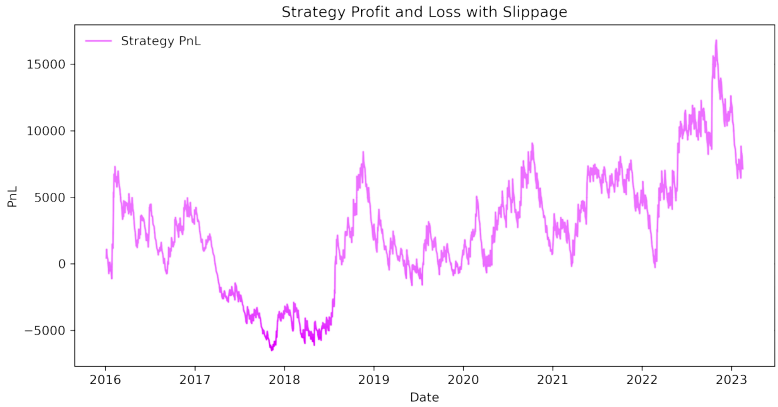


Figure 10: Strategy Equity Curve with Slippage Costs

5.4 Transaction Costs

In addition to slippage, we account for transaction fees. On average, brokers like Interactive Brokers (IBKR) charge $3 per $100,000 traded in QQQ. With the strategy scaling to hundreds of thousands per trade, these commissions further erode profitability. Once included, the final equity curve becomes flat or slightly negative, as depicted in Figure 11.

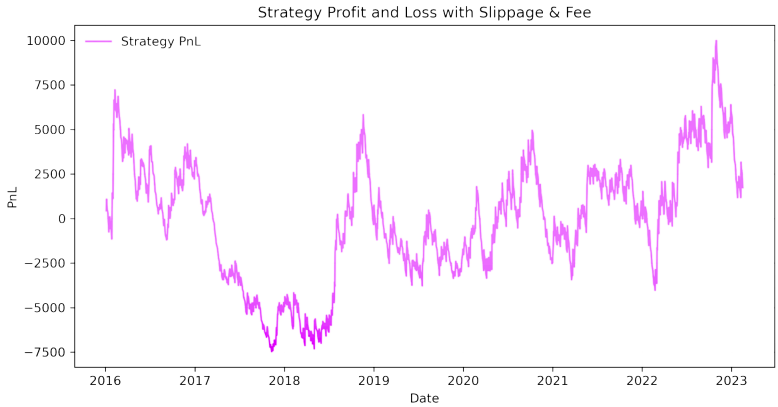


Figure 11: Strategy Equity Curve with Slippage and Commission Costs

5.5 Conclusion

Although the strategy appears robust and highly profitable on paper, particularly when backtested under idealized conditions, it suffers from major limitations when evaluated under realistic trading constraints:

- The money management rule leads to impractical position sizes.
- The order book of QQQ cannot accommodate such trades without significant slippage.
- The average per-trade edge is too narrow to survive even moderate frictions.
- Once slippage and fees are included, the strategy fails to generate net returns.

This case study highlights the crucial distinction between theoretical edge and real-world execution. A robust strategy must not only produce positive expected value but also remain executable under live market conditions with acceptable risk and cost. The ORB strategy, while elegant in its structure, fails to meet these criteria at scale.

6 Improving the Strategy: Futures-Based Signal Enhancement

In an effort to improve the original ORB strategy presented in the paper, we propose several modifications aimed at reducing trading frictions and increasing the signal's predictive power.

6.1 Trading the Futures Instead of the ETF

The first idea is to replace the QQQ ETF with its futures counterpart, the Nasdaq E-mini Future (NQ). This instrument offers several advantages:

- **Lower relative spread**: In percentage terms, the bid-ask spread of the NQ future is approximately half that of QQQ, thus reducing slippage-related costs.
- **Higher notional exposure per contract**: A single NQ futures contract represents roughly $440,000 of NASDAQ exposure, allowing large trades to be executed with far fewer contracts, minimizing market impact.

While this transition may help mitigate costs, it alone is insufficient to make the original strategy viable due to the still-low average trade return.

6.2 Improving the Entry Signal Using Pre-Market Futures Data

The second and more impactful idea focuses on enhancing the quality of the entry signal. Since futures trade nearly 23 hours a day, unlike ETFs, which are confined to regular market hours, NQ futures provide valuable price information prior to the equity market open. This intraday lead-lag relationship can potentially be exploited.

We revise the original entry rule as follows:

- **Long entry**: Only enter a long trade if:
  1. The first one-minute candle of the QQQ ETF is positive.
  2. The price action of the NQ future during the entire pre-market session (e.g., from midnight to 9:30 AM) is also positive.
- **Short entry**: Only enter a short trade if:
  1. The first one-minute candle of the QQQ ETF is negative.
  2. The price action of the NQ future during the entire pre-market session is negative.

Applying this new rule results in the equity curve shown in Figure 12. While the number of trades decreases due to stricter entry conditions, the **average trade profit increases from 7 to 10 cents**, improving the strategy's efficiency.

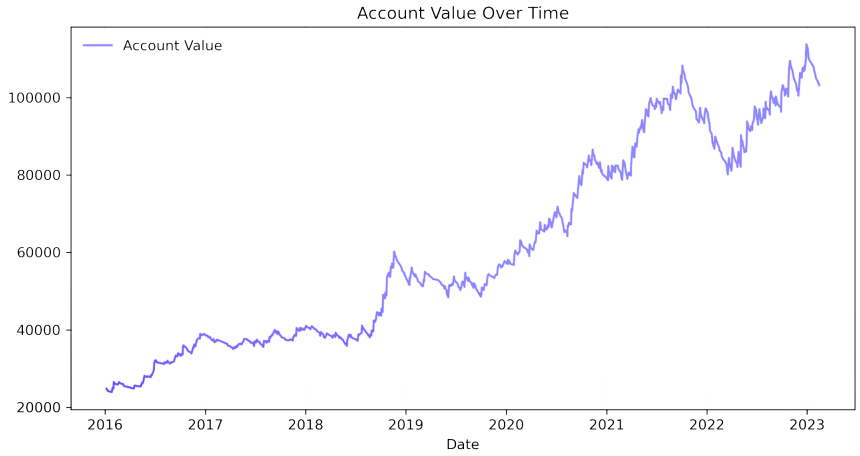


Figure 12: Strategy Equity Curve Using Pre-Market NQ Futures Trend

6.3 Refining the Signal: Focus on the Last Five Minutes Before Open

Recognizing that a full pre-market session may introduce excessive noise, we refine the signal further by narrowing the futures observation window to the final five minutes before the cash market opens (9:25–9:30 AM). This shorter window better aligns with short-term momentum dynamics.

The updated entry rule becomes:

- **Long entry**: Enter long if:
  1. The first one-minute QQQ candle (9:30–9:31) is positive.
  2. The final five-minute NQ futures candle (9:25–9:30) is also positive.
- **Short entry**: Enter short if:
  1. The first one-minute QQQ candle is negative.
  2. The 9:25–9:30 NQ futures candle is also negative.

The results of this revised strategy are illustrated in Figure 13. This more focused signal produces a smoother equity curve and raises the **average trade profit to 15 cents** — more than double the original value in the paper.

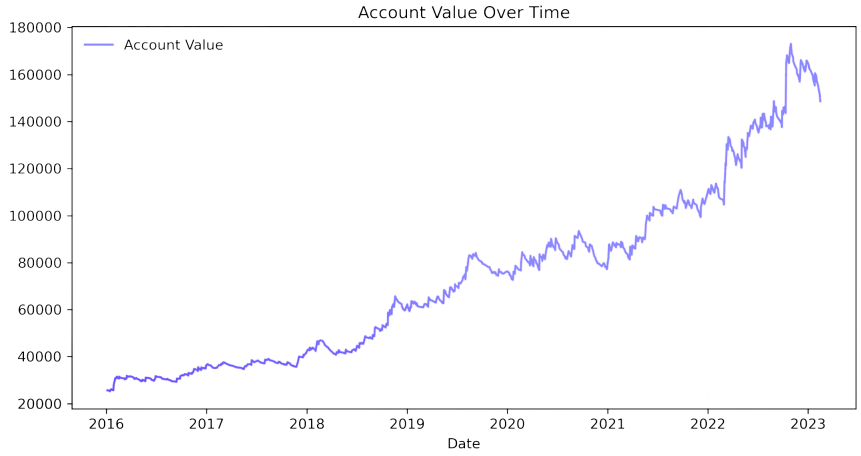


Figure 13: Strategy Equity Curve Using Final Five-Minute NQ Candle

6.4 Summary

This section demonstrates that both transaction cost reduction and signal refinement are critical when designing high-frequency momentum strategies. The use of

futures not only lowers slippage but also enables signal enhancement via pre-market price dynamics. The final iteration, based on the five-minute futures trend preceding the market open, offers the best trade-off between frequency and profitability, yielding a more stable and executable strategy in practice.

7 Conclusion

In conclusion, the original opening range breakout (ORB) strategy as presented in the paper is not immediately viable for practical trading, primarily due to its reliance on aggressive money management, high leverage, and an extremely low average trade return (approximately 7 cents). The strategy's profitability hinges almost entirely on compounding position sizes over time, which leads to impractically large trade volumes (e.g., over 3,200 QQQ shares after 8 years), making it extremely vulnerable to execution slippage and liquidity constraints. These limitations are particularly critical when trading at market open, where spreads and volatility tend to be elevated and order book depth is limited.

However, through a series of refinements, we demonstrated that it is possible to meaningfully improve the strategy's risk-return profile. First, we proposed using Nasdaq futures (NQ) instead of the QQQ ETF to both reduce slippage and enable more scalable exposure via fewer contracts. Then, we enhanced the entry criteria by incorporating information from the pre-market futures price action — initially using the full pre-market session, and later narrowing the signal to the last five minutes before the market open. This allowed us to better capture short-term momentum while reducing noise.

As a result, the average trade profit increased from 7 cents to 10 cents with the broader futures trend, and up to 15 cents when focusing on the final five-minute candle before the open. These improvements demonstrate that even simple rule-based strategies can benefit substantially from more thoughtful execution logic and smarter signal filtering.

Nevertheless, the strategy remains fragile from an execution perspective. In a professional trading environment, further enhancements could be achieved by avoiding market orders altogether and instead employing limit orders supported by liquidity engineering techniques, such as smart order routing, iceberg orders, or volume participation algorithms. These tools can reduce slippage and help ensure that fills occur at more favorable prices, especially in the critical first minutes of the trading session.

While this strategy in its current form may not be robust enough for direct im-

plementation, the process of improving it reveals valuable lessons on the importance of microstructure awareness, execution quality, and signal validation. These are crucial components of any algorithmic trading system operating at high frequency or within intraday timeframes.

Disclaimer

This research project is intended for academic and educational purposes only. The strategies and results discussed herein are based on historical data and hypothetical simulations and do not constitute financial advice or a recommendation to buy or sell any financial instrument. Past performance, including that of the enhanced strategy, is not indicative of future results. The implementation of any trading strategy in live markets involves significant risk, including the risk of capital loss, slippage, and liquidity constraints.

Readers should be aware that the strategy has not been tested under real market conditions, and many factors — such as transaction costs, latency, psychological biases, and changing market dynamics — can significantly alter its performance in practice. Professional advice should be sought before attempting to replicate or implement any trading methodology in a live environment.

End of report